Shashidhara - UI Process & Real-Time Visualization

STC89C52 Microcontroller Multi-Process Simulator - Student Study Guide

Role: UI/UX Engineer, Web Server Developer & Desktop Swing Architect

1. What is TCP in Simple Terms?

TCP is the **cable between the screen and the engine**.

Your UI runs in its own process. When the user clicks the green `[ Step ]` button:

1. The UI writes `"STEP\n"` across the TCP socket to port 5000.
2. CoreProcess executes 1 instruction and writes back `"STATE|PC=0002|A=10|..."`.
3. Your UI parses this string and updates all 6 panels instantly.

Because TCP is reliable and ordered, your UI never receives a response from step 2 before step 1!

2. Is Connecting Java to HTML Difficult for Beginners?

NO, it is actually the cleanest and easiest way.

- We did **NOT** use complex frameworks like React, Node.js, Vite, or Spring Boot.
- We used Java's built-in `com.sun.net.httpserver.HttpServer` which takes just 5 lines to start:

```
HttpServer webServer = HttpServer.create(new InetSocketAddress(8080), 0);
webServer.createContext("/", new WebHandler()); // Serves the HTML
webServer.createContext("/api/cmd", new ApiHandler()); // Forwards button clicks
webServer.start();
```

- When a user clicks `[ Step ]`, the browser calls `fetch('/api/cmd?cmd=STEP')`. Java grabs `"STEP"`, asks `CoreProcess` for the new numbers, and sends them back to the screen!

3. Files Owned & Managed by Shashidhara

- `src/UIProcess.java`: Standalone UI process embedding an HTTP web server on port 8080 and interactive CLI console.
- `src/SimulatorUI.java`: Full-featured Java Swing desktop GUI connecting directly to CoreProcess over TCP socket.

4. Flowchart: UI Dual-Interface Architecture

5. The 6-Panel Real-Time Dashboard

1. **Panel 1: Assembly / Hex Opcode Editor:** Input custom hex opcodes (e.g. `74 FE 24 03 ...`) with preset buttons (`Preset 1: Math`, `Preset 2: Queue`, `Preset 3: ANL`, `Preset 4: SJMP`) and an opcode cheat-sheet.
2. **Panel 2: Execution Trace:** Shows the three distinct machine phases:

- `Fetch`: Opcode byte loaded from PC.
- `Decode`: Opcode mnemonic identified.
- `Execute`: State mutation details and flag changes.

3. **Panel 3: Registers & PSW Flags:** Displays `PC`, `SP`, `ACC`, `B`, register bank `R0-R7`, and color-coded chips for `CY`, `AC`, `OV`, and `P`.
4. **Panel 4: Hardware Stack:** Real-time stack pointer, top-of-stack memory value, and current pushed depth.
5. **Panel 5: Circular FIFO Queue:** Real-time size, capacity (16), front pointer, rear pointer, and queued elements array.
6. **Panel 6: 256-Byte Memory Inspector:** Hexadecimal matrix showing all 256 bytes of internal RAM with live updates.

---

6. Desktop Swing GUI (`SimulatorUI.java`)

- If your professor asks for a traditional textbook Java GUI, run: `java -cp src SimulatorUI`.
- **Event Dispatch Thread (EDT) Safety:** All Swing UI updates run inside `SwingUtilities.invokeLater()` to prevent threading violations and UI freezes.
- **Color-Coded Visual Indicators:** Registers light up when their values change between steps (e.g., Accumulator transitions from `00` to `FE`).

---

7. How Shashidhara Can Add Features as College Teaches More

- **Adding a New Button (e.g. `[ Clear Memory ]`):**

In `src/UIProcess.java`, add the HTML button: `<button onclick='send("CLEAR")'>[ Clear ]</button>`.

In `ApiHandler`, add `"CLEAR"` to `WEB_COMMANDS`.

- **Adding a New Display Metric:**

Add a `<div>` with an id like `<span id='cycle-count'>0</span>`.

In `render(s)`, add `set('cycle-count', s.CYCLES);`.

---

8. Shashidhara's 5 Hostile Examiner Questions & Defenses

Q1: "What happens in your UI if CoreProcess is not running or crashes?"

Answer: *"In `UIProcess.java`, socket connection attempts are wrapped in a try-catch block. If CoreProcess is offline, the status indicator turns red, the status bar displays 'Core offline - start CoreProcess on port 5000', and all data panels gracefully display '--' rather than crashing."*

Q2: "How does the Web UI work without any external frameworks or internet?"

Answer: *"We use standard Java standard library `com.sun.net.httpserver.HttpServer` and pure embedded HTML, CSS, and vanilla JavaScript. There are zero external dependencies, no node_modules, and no CDN links, so the UI functions 100% offline in any air-gapped lab environment."*

Q3: "How does the UI parse the state packet sent by CoreProcess?"

Answer: *"Core sends a pipe-delimited string: `STATE|PC=0002|A=10|CY=false...`. In JavaScript and Java, we split on `|`, find each key-value pair separated by =, store them into a hash map, and populate the corresponding DOM elements."*

Q4: "Why did you build both a Web interface and a Swing Desktop GUI?"

Answer: *"To demonstrate separation of concerns. CoreProcess implements an open, standardized socket protocol. Any client that speaks this protocol--whether a browser HTTP proxy, a Swing desktop application, or a simple command-line terminal--can inspect and drive the simulator without changing a single line of CPU code."*

Q5: "How do you handle rapid button clicks on the web dashboard?"

> ***Answer:*** *"Each button triggers an asynchronous* `fetch()` *API request to* `/api/cmd?cmd=STEP`*. The browser awaits the response before rendering state changes, preventing command overlap."*